Delphi Web Studio Pro Capabilities and Publishing Reference

Google keys, website compatibility, comparisons, GitHub, credits, and future translation

Last changed: November 9, 2026

Based on the current application source and the official external references listed in this document.

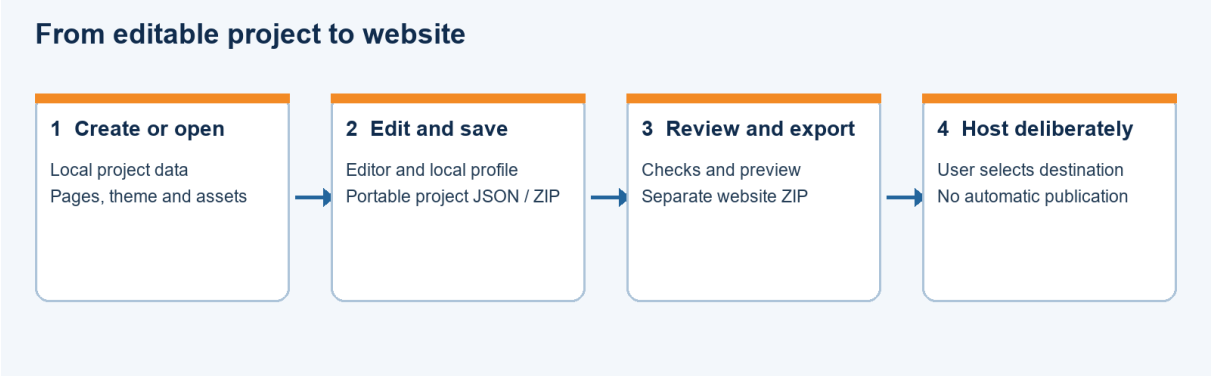


Figure 1. Local project work and the separate hosting decision.

Contents

4 Google Gemini key....3
5 Website and hosting compatibility....4
6 Comparison with Wix....4
7 Comparison with WordPress....4
8 GitHub website setup....4
9 GitHub push and publication....5
10 Authors and other credits....5
11 Future Delphi App Translation integration....5
Sources....6
Current editor interaction notes....6
Static hosting options for parish websites....7
Recent editor and export capabilities....8

4 Google Gemini key

A Gemini API key is optional. The app can create local starters and edit projects without a Google key. A key is used only for the optional cloud generation path, and sending a request requires the relevant session consent. Local AI is a different feature that uses separately selected local runtime and model files.

1. Open Google AI Studio at https://aistudio.google.com/ and sign in. Accept the applicable terms. A new user may be given a default Cloud project and key; existing users may need to import an existing project.
2. Open the API Keys page, select an appropriate project, and choose Create API key if a suitable key does not already exist. Copy the new key privately. If creation is unavailable, check project permissions and account eligibility in Google's documentation.
3. In Delphi Web Studio Pro, open Settings, choose API Keys, enter the Gemini key, and enable cloud generation for the current session when you intend to use it. Use Test if desired. A successful key test establishes that the endpoint accepted that test; it does not guarantee model access, quota, billing, or successful generation.
4. Review any prompt before sending it and review the generated content before accepting it. Do not include secrets or personal information unless you intend to send them to the service.

How long does the key last? Google does not give one universal lifetime for every key and policy. A key can be restricted, revoked, blocked, or otherwise stop working; check its current status in AI Studio. Google is moving Gemini access to authorization keys, so use a currently supported key and check Google's latest guidance rather than relying on an old standard key.

Must it be re-entered? Newly entered keys and cloud consent are session-only in this app: after the app closes or reloads, enter the key and consent again for another session. This is an application storage behavior, not proof that Google issued a new key each time. An older key from a previous version may remain in local storage until removed using the available control.

Treat a key like a password. Keep it out of project files, exports, screenshots, and public repositories. If exposed, replace and revoke it through Google. Quotas, availability, and charges depend on the project, chosen service, and current Google terms.

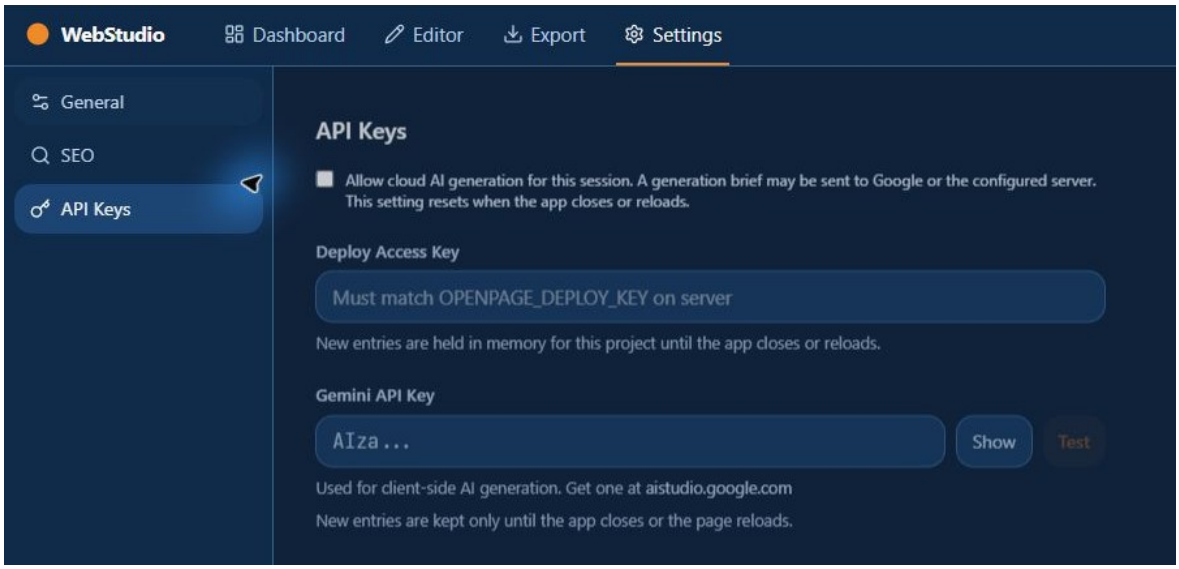


Figure 2. The API Keys settings screen.

5 Website and hosting compatibility

The application creates editable project files and can export static HTML or a website ZIP containing pages and assets. A compatible static host can serve the exported files when the expected paths and any subfolder settings are correct. Test the extracted result on a local web server and then on the intended host before making it public.

This is not a guarantee that the output can be dropped into any existing site or any site builder. Host restrictions, relative paths, custom domains, browser policies, external media, JavaScript modules, and server configuration affect behavior. Forms and newsletter features need an appropriate service; static hosting does not itself provide a form-processing server. Keep the project separately from the exported site.

6 Comparison with Wix

Both products offer visual website construction with reusable sections and page styling. Delphi Web Studio Pro is a local experimental Windows editor that produces static files for an independently chosen compatible host. Wix is a hosted website platform with its own integrated editor, infrastructure, and service features.

They are not interchangeable project formats. Wix states that a Wix site relies on Wix infrastructure and must run on its servers for proper operation. Delphi Web Studio Pro does not claim to open or faithfully reproduce an arbitrary Wix site. Wix includes hosted features outside the scope of this application; assess the specific features and costs that matter for a particular site.

7 Comparison with WordPress

Both can organize pages and article-like content. WordPress is a full content management system with themes and plugins, generally running on a web server. Delphi Web Studio Pro edits a local project and produces static output; it is not a WordPress installation and does not run WordPress plugins, server-side PHP, accounts, or database features.

The app has a limited WordPress XML import for selected content. It is a conversion, not a theme or plugin migration. Review the report, pages, images, links, drafts, and retained source; keep the original WordPress export. The app does not currently export a complete WordPress theme or guarantee that a WordPress site can be recreated.

8 GitHub website setup

The current application does not automatically create a GitHub account, repository, GitHub Pages site, or required settings for a user or organization. A user or administrator must arrange the GitHub account, permissions, repository, and Pages configuration separately. GitHub’s documentation explains current eligibility and configuration; availability can depend on account plan and repository visibility.

The exported website can be placed in an appropriate GitHub Pages publishing source after the owner deliberately configures it. A project site may live under a repository subpath; set and test the site URL and paths accordingly. GitHub Pages serves static files and does not run server-side PHP, Ruby, or Python. Configuring or enabling a public site is a publication decision and has not been done for this project.

9 GitHub push and publication

The current local editor exports or reviews files; it does not automatically push a site to GitHub or automatically publish it to GitHub Pages. The GitHub link in the editor points to the application repository; it is not a setup or deployment wizard for the user's site.

A user must take separate action to transfer the intended output into a configured repository. Once a GitHub Pages publishing source has been deliberately enabled, GitHub can automatically publish subsequent eligible pushes to that source; GitHub's behavior is separate from this app. Review the destination and files before each push. Do not configure or publish a site until the owner explicitly wants that action.

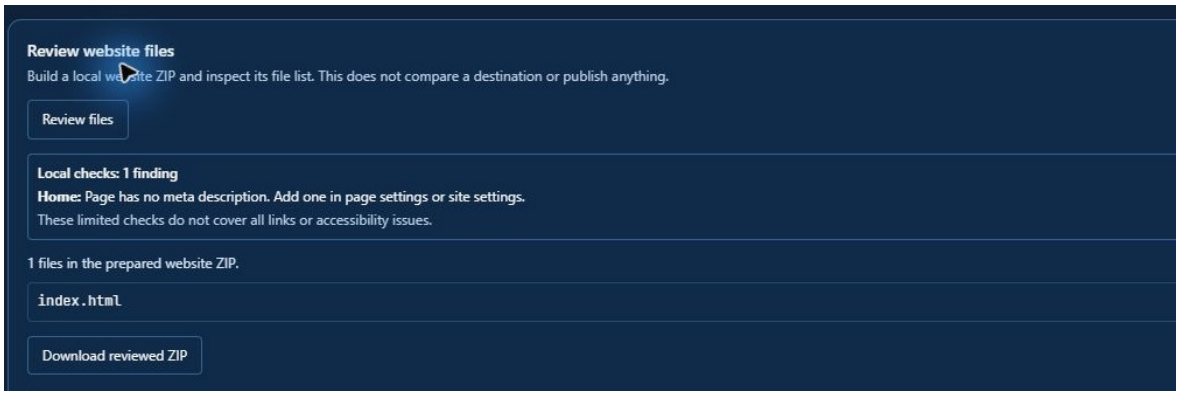


Figure 3. Local review is separate from upload and publication.

10 Authors and other credits

For an article, select a non-Home page, open Articles, mark it as an article, and enter the author and optional summary and tags. A ready article can show its author in the generated page and article index. A draft remains in the project but is excluded from the generated site until ready.

For site-wide credits, select the Footer component and use its Copyright field, description, or a normal content section or credits page. Enter names, permissions, image credits, and license notices as appropriate. These fields are user-authored text; the app does not automatically determine ownership or verify a license.

11 Future Delphi App Translation integration

A later integration is possible in principle, but it is not installed or verified here. Delphi Web Studio Pro has two different user interfaces: a Delphi VCL host with designer-backed forms and dialogs, and a React/TypeScript editor displayed in WebView2. The separate Delphi App Translation Studio project has a Delphi-oriented runtime and form-scanning approach. It should be assessed against a copy of the VCL forms and Delphi strings first.

A practical plan is to inventory native form properties, menus, dialogs, and resourcestrings; scan the project with the translation tool; prepare and review language packs; add the designer-supported VCL manager and language selector where appropriate; and test switching, persistence, layout, accelerators, and placeholders. The other project's documentation states that resourcestring entries may need explicit translation calls, so automatic coverage should not be assumed.

The React editor's TypeScript strings and the content of exported websites need a separate localization approach and language resources. Translating the native shell alone will not translate the editor or a user's

website. Preserve product names and user content, test right-to-left layouts if supported, and evaluate version compatibility and licensing before integration. This is a future proposal, not work already performed.

Sources

Application evidence: src/routes/Settings.tsx; src/lib/generate-site.ts; src/routes/Deploy.tsx; src/lib/export-html.ts; src/editor/ArticlesPanel.tsx; desktop/DelphiWebStudio; and the User Guide. Future-translation assessment also reviewed the separate Delphi App Translation V2 project documentation and source layout.

Google Gemini API keys: https://ai.google.dev/gemini-api/docs/api-key

Google key troubleshooting: https://ai.google.dev/gemini-api/docs/troubleshooting

Google billing: https://ai.google.dev/gemini-api/docs/billing

GitHub Pages setup: https://docs.github.com/en/pages/getting-started-with-github-pages/creating-a-github-pages-site

Wix hosting and export: https://support.wix.com/en/article/can-i-host-a-wix-site-on

WordPress features: https://wordpress.org/about/features/

External services change; these references and this app's implementation should be checked again before relying on a particular provider setup.

Current editor interaction notes

The editor provides horizontal and vertical decorative line presets with selectable style, thickness, and color. It offers movable section objects, panels, a Layers list, overlapping-item selection, and group movement for section objects. These capabilities are not a claim of full Wix feature or interaction parity. Check the active page in desktop, tablet, and phone previews before export.

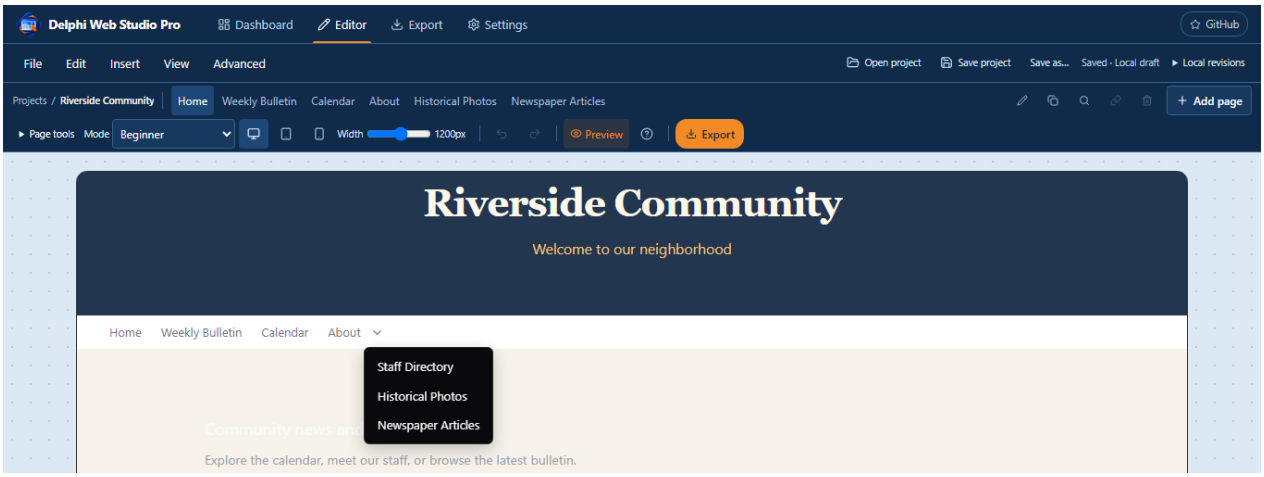
Static hosting options for parish websites

Use the website ZIP containing index.html, page files and assets for a static host. Keep the editable project ZIP separately. For example, a parish may host the public export on Cloudflare Pages, GitHub Pages or Netlify and use its own domain. Cloudflare Pages accepts the website ZIP through Direct Upload and limits individual assets to 25 MiB. GitHub Pages can serve an owned domain in place of a github.io address. Review current host limits and DNS requirements before any change.

Cloudflare Pages gives a pages.dev address and can map www through a CNAME at the current DNS provider. Using the bare domain with Cloudflare Pages requires Cloudflare nameservers. Netlify accepts the extracted output folder and includes a custom domain with HTTPS on its free plan. Its usage credits cover production deployments and traffic. GitHub Pages requires a public repository on GitHub Free and supports owned root domains and subdomains.

A static host serves navigation, calendars, galleries, staff biographies and PDFs from the website export. It does not supply a contact form mail backend automatically. Test only with a deliberately configured safe destination. Preview and local website review are separate from any publishing action.

Provider references checked November 9, 2026. developers.cloudflare.com/pages/get-started/direct-upload/ · developers.cloudflare.com/pages/configuration/custom-domains/ · docs.github.com/en/pages/configuring-a-custom-domain-for-your-github-pages-site/about-custom-domains-and-github-pages · docs.netlify.com/deploy/create-deploys/

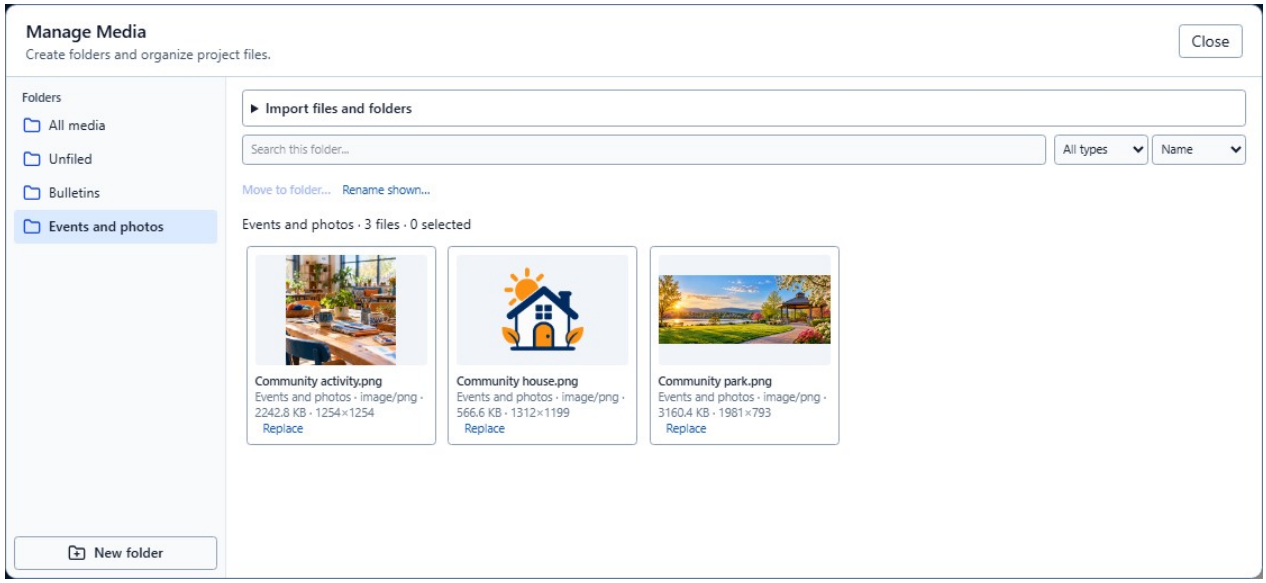


About stays clickable. Its separate arrow opens the submenu in Preview.

Recent editor and export capabilities

The current editor supports independent page tabs with fixed commands, project media folders and a separate Manage Media window. Responsive galleries, a local monthly calendar, staff biographies in a scrollable reader and Navbar submenus work in the editor's Preview and in generated website HTML.

For a church archive, use the responsive Gallery for historical photographs and individual Image blocks for tall newspaper scans. Place scans in the intended order, add source dates, and link the page from the About submenu. Moving media between project folders preserves existing page references.



Manage Media organizes files in this project. Select a folder in the sidebar to see its contents.